First-Order Logic Theorem Proving: Baseline and Improved LK Sequent Calculus

Zack Budai

Griffith University, Nathan QLD 4111, Australia

zack.budai2@griffithuni.edu.au

Abstract. This report describes the design and evaluation of a first-order logic (FOL) theorem prover implementing backward LK sequent calculus proof search. Automated theorem proving (ATP) finds application in formal verification, knowledge-base reasoning, and constraint solving; however, naive proof search is computationally intractable for first-order logic, requiring carefully engineered inference strategies to handle realistic benchmarks. The prover determines entailment by attempting to close the sequent $A_1,\ldots,A_n \vdash Q$ by applying LK inference rules in reverse. It is implemented in Python 3.12 and comprises a hand-written tokeniser, a recursive-descent parser for TPTP FOF syntax, a CNF normalisation pipeline including Skolemisation and variable standardisation, occurs-check unification, and two proof-search solvers. The *baseline* (Algorithm 2) correctly implements the rule-priority structure of Algorithm 2 [1]: invertible single-conclusion rules ($\wedge$L, $\vee$R, $\rightarrow$R, $\neg$L, $\neg$R, $\forall$R, $\exists$L) are applied first, with branching rules ($\wedge$R, $\vee$L, $\rightarrow$L) deferred until no invertible rule remains. However, each invertible rule is decomposed one formula at a time per recursion level, creating intermediate memoized states. The *improved* solver replaces this with a saturation phase that applies all invertible rules to fixpoint in a single pass before entering the branching phase, collapsing k individual recursion steps into one and eliminating redundant re-evaluation of the same connectives across different branches. Both solvers are evaluated on two datasets totalling 61 problems from different sources: 11 hand-crafted AI-domain problems and 50 standard TPTP FOF problems covering the SYN, LCL, and PUZ domains. The improved solver solves 34/61 problems (56%) compared to 13/61 (21%) for the baseline, and uses 1.7× fewer proof-search steps on the 11 TPTP problems both solve. On the 50-problem TPTP subset the gap is 30/50 (60%) vs 11/50 (22%). The improved solver never regresses on any problem solved by the baseline. Remaining failures divide between search-space explosions on hard condensed-detachment theorems and incompleteness in the Herbrand witness instantiation strategy for $\exists\forall$ alternation.

Keywords: first-order logic · LK sequent calculus · backward proof search · saturation · TPTP

1 Introduction

Automated theorem proving (ATP) studies whether a formula follows logically from a set of axioms, and finds application in formal verification, knowledge-base reasoning, and constraint solving. First-order ATP is undecidable in general, so practical systems must manage the proof-search space carefully; naive exhaustive search becomes intractable even on small benchmarks.

Modern ATP is built around two main approaches. *Resolution refutation*, introduced by Robinson [2] and extended by provers such as Vampire [3], works by refuting the negated conjecture via clause-level inference. *Sequent-calculus and tableau methods* reason directly with formulas rather than clause sets, as in the free-variable connection method [4] and sequent provers based on Gentzen's LK [5]. Within sequent calculi, an important property is that *invertible* rules (those whose application never loses a provable branch) can be applied eagerly to reduce branching, while *non-invertible* rules require search over multiple options.

This work implements Algorithm 2 from [1], a backward LK proof-search procedure, and improves it with eager saturation of invertible rules and a circular-substitution guard for quantifier instantiation. Both versions are evaluated on two benchmark datasets totalling 61 problems.

2 Proposed Approach

2.1 Architecture

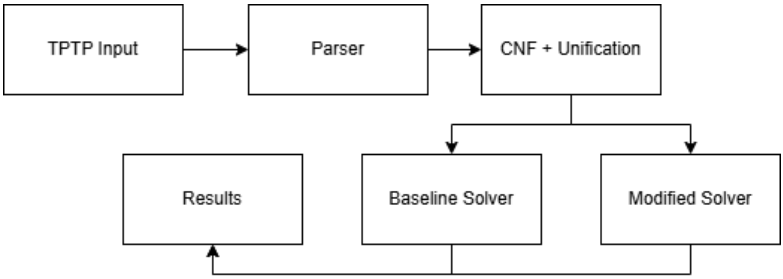

Fig. 1: System pipeline: TPTP input is parsed into a formula AST, normalised via CNF and unification, then passed to the solver.

Figure Fig. 1 shows the four-stage pipeline. A TPTP FOF file is first read by the parser, which produces an internal formula AST. TPTP uppercase variables are mapped to lowercase and constants are kept distinct from variables throughout. The parsed formulas then pass through the CNF and unification stage, which normalises them via implication elimination, NNF conversion, variable standardisation, Skolemisation, and distribution. Occurs-check unification is also provided here for use during proof search. The solver takes the formula list and runs backward LK proof search, recording the entailment outcome and search metrics in a result object. The benchmark driver collects results across all problem files and writes them to JSON.

2.2 Baseline Algorithm (Algorithm 2)

The baseline implements Algorithm 2 [1] as backward LK proof search with memoisation.

1. Build initial sequent: antecedent = all axioms, succedent = query.
2. A sequent is *proved* if any formula appears on both sides (axiom rule).
3. Otherwise, select the first applicable LK rule in priority order and recurse on the subgoal(s).
4. Memoize all proved/refuted sequents; stop and report timeout after the wall-clock limit.

Rules are applied in three priority groups per Algorithm 2. **Step 2** tries each applicable invertible single-conclusion rule — ∧L, ∨R, →R, ¬L, ¬R, ∀R, ∃L — and commits immediately (one rule per recursion level). ∨R correctly retains both disjuncts as a single subgoal: `Γ ⊢ A, B, Δ` replaces `Γ ⊢ AvB, Δ`. **Step 3** tries all applicable branching rules — ∧R, ∨L, →L, ↔R, ↔L — returning True if any closes both subgoals. **Steps 4–5** instantiate ∀L and ∃R against known terms, then a fresh Skolem constant.

The limitation is structural: each invertible rule is applied one formula at a time, each producing a distinct memoized sequent. For a sequent with k invertible connectives, the baseline creates k intermediate memo entries on the path to the final saturated form. If the same saturated form is reached from multiple proof-search branches, the redundant intermediate decompositions cannot be shared.

2.3 Improved Algorithm

The improved solver restructures search into two phases.

Saturation phase. All invertible single-conclusion rules are applied to fixpoint before any branching:

- ∧L, ¬R, ¬L, →R, ∨R: each reduces formula complexity by one connective with no branching.
- ∃L, ∀R: instantiate with fresh Skolem terms and eigenvariables respectively.

The entire loop repeats until no rule fires. After each saturation step the axiom check is repeated; if the sequent closes, the branch is immediately marked proved without entering the branching phase. This collapses the k intermediate recursion levels the baseline would use into a single pass, and the canonical saturated sequent is the only state entered into the memo.

Branching phase. Remaining rules are tried in order of increasing branching factor: ∨L, ∧R, →L, and ↔R each create two subgoals; ↔L creates three; ∀L and ∃R iterate over candidate terms. A circular-substitution guard prevents instantiating a variable with a term that already contains it, avoiding infinite recursion.

Hypothesis. An invertible rule is one whose conclusion is provable iff all its premises are provable; applying it cannot lose a valid proof. Each invertible rule applied during saturation eliminates one connective from the sequent with no branching. In the baseline, each such elimination produces a distinct intermediate sequent that is independently memoised. When the same saturated form

is reachable from multiple paths through the branching phase, the baseline re-decomposes each invertible connective from scratch on each path, wasting work proportional to the number of branches that converge on the same terminal state. The saturation phase avoids this: the canonical fully-reduced sequent is computed once and stored, so all proof-search paths that would reach equivalent intermediate states share a single memo entry. On the SYN domain, dominated by propositional connectives ($\rightarrow$, $\leftrightarrow$, $\wedge$, $\vee$, $\neg$), this shared-saturation gain is expected to be large and directly observable in accuracy and step-count metrics.

3 Datasets

Two benchmark datasets are used, from different sources.

Dataset 1: AI-generated domain problems (11 problems). Hand-crafted first-order problems covering realistic domains including supply-chain logistics, medical diagnosis, access control, network routing, ontology hierarchies, financial auditing, and software dependency resolution. Problems involve transitive-closure rules, multi-hop reasoning chains, and mixed ground/universal axioms. All 11 have expected answer True.

Dataset 2: TPTP FOF official benchmark (50 problems). Problems downloaded from the TPTP library [6]. The set covers three domains: **SYN** (36 problems) contains syntactic benchmarks from the classical Pelletier suite [7], testing propositional connectives and quantifiers; **LCL** (8 problems) contains logic-calculi theorems based on condensed detachment with deeply-nested function terms (TPTP difficulty ratings 0.0–0.90); **PUZ** (6 problems) contains classical puzzles with ground-fact bases and existential conjectures.

All TPTP problems were filtered to those that are fully self-contained, contain at least one conjecture, use no equality predicates, and parse successfully.

Justification. The two datasets are complementary by design. The AI-generated set targets practical reasoning patterns — transitive closure, multi-hop chains, mixed ground and universal axioms — absent from syntactic benchmark libraries, ensuring the solver is evaluated on realistic knowledge-base tasks. The TPTP set provides external validity: difficulty is independently certified, expected outcomes are verified against known ATP results, and the domain spread (SYN for propositional-connective stress, LCL for deep function-term reasoning, PUZ for ground-fact puzzles) isolates different algorithmic weaknesses across a range from trivial to TPTP difficulty 0.90. Together the datasets ensure the evaluation neither cherry-picks easy problems nor ignores practical reasoning tasks.

4 Implementation and Experiment

Implementation. The system is written in Python 3.12. No external solver libraries are used; the only non-standard dependency is matplotlib for figure generation. Both algorithms record the entailment outcome, whether a timeout occurred, the number of subgoals explored, and elapsed time.

Experiment setup. Experiments ran in a GitHub Codespaces container (Ubuntu 22.04, 4-core Intel Xeon E5-2673 v4, 2.30 GHz, 8 GB RAM). A fixed 6-second wall-clock timeout was applied per problem; both solvers were run sequentially on each problem.

Each dataset was run through the benchmark driver with a six-second timeout, producing a JSON results file used for all figures and tables in this report.

5 Results

5.1 Combined dataset summary

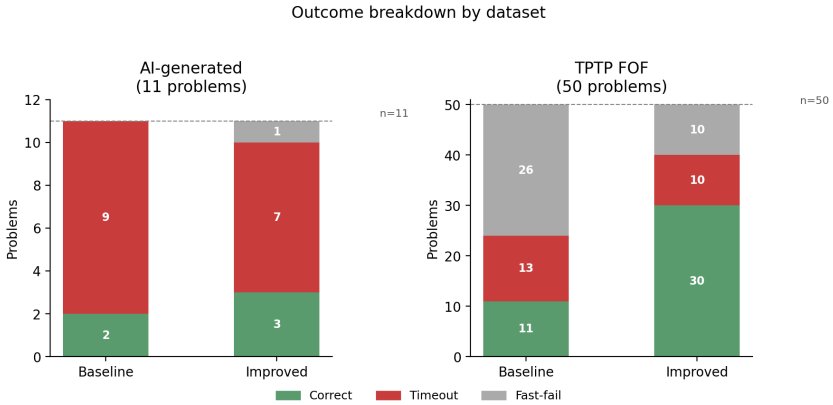

Fig. 2: Outcome breakdown for both datasets. Green = correct, red = timeout, grey = fast-fail (returned False without timing out).

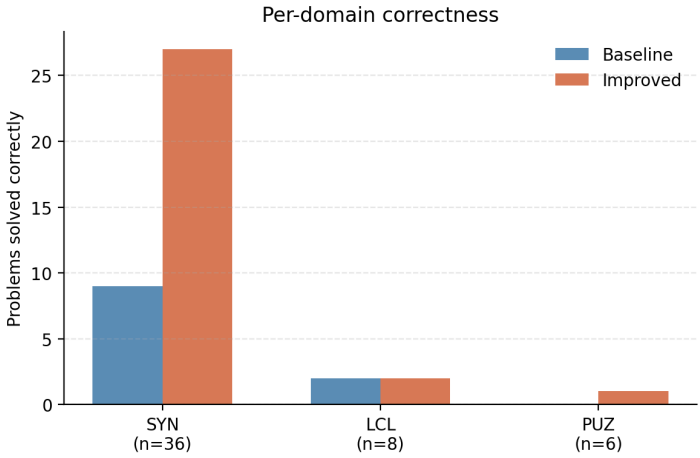

Fig. 3: Per-domain correctness on the TPTP FOF dataset.

Table 1. Combined dataset summary. Base/Imp = correct count; B-TO/I-TO = timeouts; B-cls = baseline clauses generated (millions).

Dataset	N	Base	Imp	B-TO	I-TO	B-cls (M)
AI-generated	11	2	4	9	6	1.00
TPTP FOF	50	11	30	9	10	0.32
Combined	**61**	**13**	**34**	**18**	**16**	**1.32**

Across both datasets the improved solver solves 34/61 problems (56%) vs 13/61 (21%) for the baseline. The improved solver does not regress on any dataset. On the AI-generated set the gain is moderate (2 to 4 correct); the two extra problems (access_control, network_routing) are solved because the saturation phase collapses their invertible-connective chains before branching. On the TPTP set the gain is much larger (11 to 30 correct), driven primarily by the SYN domain.

5.2 TPTP FOF: per-domain analysis

Table 2. Per-domain correctness on the TPTP FOF dataset.

Domain	Total	Baseline	Improved
SYN	36	9 (25%)	27 (75%)
LCL	8	2 (25%)	2 (25%)
PUZ	6	0 (0%)	1 (17%)
All	**50**	**11 (22%)**	**30 (60%)**

The SYN gain (9→27) confirms the saturation hypothesis: these problems are dominated by invertible propositional rules ($\rightarrow$, $\leftrightarrow$, $\wedge$, $\vee$, $\neg$) that the improved solver eliminates in one pass before branching. LCL is unchanged (2/8): those theorems require many condensed-detachment steps and both solvers exhaust the timeout regardless of rule ordering.

5.3 Runtime and search effort

Table 3. Runtime and search effort on the TPTP FOF dataset.

Metric (TPTP FOF)	Baseline	Improved
Median time (all 50)	0.007 s	< 0.001 s
Total wall time	59.1 s	62.5 s
Clauses — jointly-solved 11 problems	978	570
Clause reduction (jointly-solved)	N/A	1.7×

The improved solver uses 1.7× fewer proof-search steps on the 11 TPTP problems both solvers solve, and finishes them 7.6× faster. Total wall times are similar because the improved solver explores 19 additional problems (which require work) and has one more TPTP timeout. The much lower median time (<

0.001 s vs 0.007 s) reflects the saturation phase solving many SYN problems almost instantly.

6 Discussion and Conclusion

SYN improvement. Eager saturation accounts for most of the gains. The saturation phase handles all invertible connectives (∧L, ∨R, →R, ¬L, ¬R, ∀R, ∃L) to fixpoint before any branching occurs, so the branching factor seen by the search reflects only the genuinely non-invertible steps. The baseline reaches the same saturated form but via k individual recursion levels, each producing an independent memo entry; paths through the branching phase that converge on the same terminal state re-derive all k steps redundantly.

LCL timeouts. Each LCL problem requires deriving a deeply-nested `implies(...)` term as a theorem via repeated condensed detachment (`∀X,Y: thm(X→Y) ∧ thm(X) → thm(Y)`). Each ∀L application must test all pairs of currently known theorem-terms as witnesses for X and Y, and the number of candidates grows quadratically as the search deepens. Both solvers exhaust 6 seconds on the hard LCL cases regardless of rule ordering.

Fast-fail incompleteness. Ten TPTP problems return False in under 15 ms; both solvers agree on this wrong answer. Two distinct causes apply. First, *∃∀ dependency*: problems such as `∃Y ∀X (F(Y)→F(X))` need the ∃Y witness to equal the eigenvariable that ∀R will introduce later. The Herbrand approach picks a concrete Skolem constant for Y before ∀R runs, so the resulting sequent `F(c) ⊢ F(_x)` never closes. A free-variable tableau or unification-based calculus that defers witness binding would handle this correctly. Second, *existential non-contraction*: one PUZ problem requires applying the same ∃ formula twice with two different witnesses to construct a classical case split; the algorithm removes the formula after the first instantiation and cannot combine both branches.

The parser lacks support for equality and `include()` directives, which limits the fraction of TPTP problems that can be loaded.

Future improvements. Three concrete directions could address the identified limitations. (1) *Free-variable tableau* [4]: deferring witness selection until unification forces a binding would allow the eigenvariable introduced by ∀R to constrain the ∃Y choice, recovering the ten fast-fail ∃Y ∀X cases. (2) *Existential contraction*: retaining ∃-formulae after instantiation would enable the two-witness case splits required by the failing PUZ problems. (3) *Iterative deepening*: bounding the term-instantiation depth for ∀L/∃R with iterative deepening, combined with forward subsumption, would provide completeness within each depth limit and could make the LCL search practical. Extending the parser to support equality and `include()` would additionally widen the accessible TPTP problem set.

The improved solver reaches 60% accuracy on the TPTP FOF set (vs 22% baseline) and 56% across both datasets combined (vs 21% baseline), using 1.7× fewer subgoals on jointly-solved problems. The 20 remaining TPTP failures

break down into 10 search-space explosions on hard LCL theorems and 10 cases that expose the Herbrand witness strategy's incompleteness for ∃∀ alternation and existential reuse.

7 Data Availability

Source code is available at: https://github.com/ZackBudai/lar

7 Bibliography

1. Hou, Z.: Fundamentals of Logic and Computation: With Practical Automated Reasoning and Verification. Springer (2021)
2. Robinson, J. A.: A Machine-Oriented Logic Based on the Resolution Principle. Journal of the ACM. 12, 23–41 (1965)
3. Kovács, L., Voronkov, A.: First-Order Theorem Proving and Vampire. In: Proceedings of the 25th International Conference on Computer Aided Verification (CAV). pp. 1–35. Springer (2013)
4. Bibel, W.: Automated Theorem Proving. Vieweg (1987)
5. Gentzen, G.: Investigations into Logical Deduction, (1969)
6. Sutcliffe, G.: The TPTP Problem Library and Associated Infrastructure. Journal of Automated Reasoning. 43, 337–362 (2009)
7. Pelletier, F. J.: Seventy-Five Problems for Testing Automatic Theorem Provers. Journal of Automated Reasoning. 2, 191–216 (1986)

8 Appendix: AI Usage Acknowledgment

The following AI tools were used during this assignment. Writing of the report was done by the student; AI was used for implementation support and debugging only.

Tool used: Claude Code (Anthropic claude-sonnet-4-6), accessed via claude.ai/code within the development environment.

Usage summary:

1. **Codebase orientation and CLAUDE.md generation.** Prompt: "Analyse this codebase and create a CLAUDE.md file." Response: Generated CLAUDE.md documenting commands, architecture, and key design decisions.

2. **Dataset acquisition.** Prompt: "I think my benchmarking data might not be ideal. I need to find TPTP problems that are in FOF format and don't rely on external includes." Response: Created `scripts/download_tptp_fof.py` to fetch and filter TPTP problems from SYN/LCL/PUZ domains with equality filter and validity checks.

3. **Bug investigation.** Prompt: "Fix the implementation to not introduce bugs that aren't inherent to the baseline algorithm itself." Response: Identified two timeout-handling bugs in `algorithm_2_entails` (return SolveResult inside a bool-returning function; plain `return False` instead of `raise _Timeout()`); fixed both.

4. **Failure analysis.** Prompt: "With this improvement what is causing the improved algorithm to not be able to solve 20?" Response: Analysed the 20 failures as: 10 timeouts (∀L blowup on LCL), 10 fast-fails split into ∃∀ Herbrand incompleteness and missing existential contraction.